## SPRING MVC BASED QUESTIONS

---

### 1) What is Spring MVC?

Spring MVC is a web framework built on the Model-View-Controller design pattern that helps develop web applications by separating business logic, presentation logic, and request handling.

It is part of the Spring Framework and uses DispatcherServlet as the front controller.

---

### 2) What is MVC pattern?

MVC stands for:

Model → Data and business logic
View → UI (HTML/JSP/Thymeleaf)
Controller → Handles HTTP requests and connects Model with View

Purpose:
To separate concerns and make application maintainable.

---

### 3) What is DispatcherServlet?

DispatcherServlet is the front controller in Spring MVC.

It:

• Receives all incoming HTTP requests
• Finds appropriate controller
• Executes controller method
• Returns response

Without DispatcherServlet, Spring MVC cannot process requests.

---

**4) What is the role of @Controller?**

@Controller marks a class as a Spring MVC controller.

It tells Spring:
"This class contains request-handling methods."

---

**5) Difference between @Controller and @RestController?**

@Controller:

- Returns view name
- Used in MVC applications

@RestController:

- Returns JSON or data
- Equivalent to @Controller + @ResponseBody

Use @RestController when building APIs.

---

**6) What is @RequestMapping?**

@RequestMapping maps HTTP requests to handler methods.

It can be used at:

- Class level
- Method level

---

**7) Difference between @GetMapping and @PostMapping?**

@GetMapping:
Handles HTTP GET request.

@PostMapping:
Handles HTTP POST request.

---

**8) What is Model in Spring MVC?**

Model is used to pass data from controller to view.

Internally, it is a Map<String, Object>.

Example:
```
model.addAttribute("products", list);
```

---

**9) What is Data Binding?**

Data Binding automatically maps form fields to Java object properties.

Spring uses setters to bind form data to object.

Example:
```
@ModelAttribute Product product
```

---

**10) What is @ModelAttribute?**

@ModelAttribute:

• Binds form data to object
• Adds attribute to model

Used mainly in form handling.

---

**11) What is @RequestParam?**

Used to extract request parameter from URL.

Example:
```
products/search?name=Laptop
```

```
@GetMapping("/search")
public String search(@RequestParam String name)
```

---

**12) What is @PathVariable?**

Used to extract value from URI path.

Example:
products/edit/5

```
@GetMapping("/edit/{id}")
public String edit(@PathVariable Long id)
```

---

**13) What is @SessionAttributes?**

Used to store model attributes in HTTP session.

It keeps data across multiple requests.

Used mainly for multi-step forms.

---

**14) What is Redirect and why is it used?**

Redirect tells browser to make a new request.

return "redirect:/products/list";

Used to prevent duplicate form submission.

Follows POST-REDIRECT-GET pattern.

---

**15) What is @Valid?**

@Valid triggers validation on an object.

Works with validation annotations like:

```
@NotBlank
@Min
@Positive
```

Used for backend validation.

---

**16) What is BindingResult?**

BindingResult stores validation errors.

It must be placed immediately after @Valid parameter.

Example:
public String save(@Valid Product product, BindingResult result)

---

**17) What is ViewResolver?**

ViewResolver converts logical view name into actual file.

Example:
"product-list" → product-list.html

---

**18) What is the difference between @RequestBody and @ModelAttribute?**

@ModelAttribute:

- Used for form data
- Used in MVC

@RequestBody:

- Used for JSON data
- Used in REST APIs

---

**19) What happens if exception occurs in controller?**

Flow:

1. Exception thrown
2. DispatcherServlet catches
3. Looks for @ExceptionHandler
4. If not found → looks for @ControllerAdvice
5. If not found → default error page

---

**20) What is @ExceptionHandler?**

Handles specific exception inside controller.

Example:
@ExceptionHandler(RuntimeException.class)

---

**21) What is @ControllerAdvice?**

@ControllerAdvice is a global exception handler.

It handles exceptions across all controllers.

---

**22) What is the lifecycle of a request in Spring MVC?**

1. Client sends request
2. DispatcherServlet receives request

3. HandlerMapping finds controller
4. Controller executes
5. Model returned
6. ViewResolver resolves view
7. View renders response

---

**23) What is the difference between request scope and session scope?**

Request scope:
Data lives only for one request.

Session scope:
Data lives across multiple requests until session ends.

---

**24) How does Spring MVC perform validation?**

Spring integrates with Hibernate Validator.

Steps:

1. @Valid triggers validation
2. Validation annotations are checked
3. Errors stored in BindingResult

---

**25) What is the purpose of ResponseEntity?**

ResponseEntity allows:

• Custom HTTP status codes
• Custom headers
• Response body

Used mainly in REST APIs.

---

**26) What is the difference between forward and redirect?**

Forward:
Server internally forwards request.
URL does not change.

Redirect:
Browser sends new request.
URL changes.

---

**27) How can you handle 404 errors in Spring MVC?**

Use:

@ControllerAdvice
or
Custom ErrorController

---

**28) What are the advantages of Spring MVC?**

• Clear separation of concerns
• Easy validation
• Flexible configuration
• REST support
• Integration with Spring ecosystem

---

**29) How is Spring MVC different from Spring Boot?**

Spring MVC:
Web framework.

Spring Boot:
Auto-configuration + embedded server + rapid development.

Spring Boot internally uses Spring MVC.

---

**30) How do you test Spring MVC controllers?**

Using:

- MockMvc
- @SpringBootTest
- Postman (for REST)